



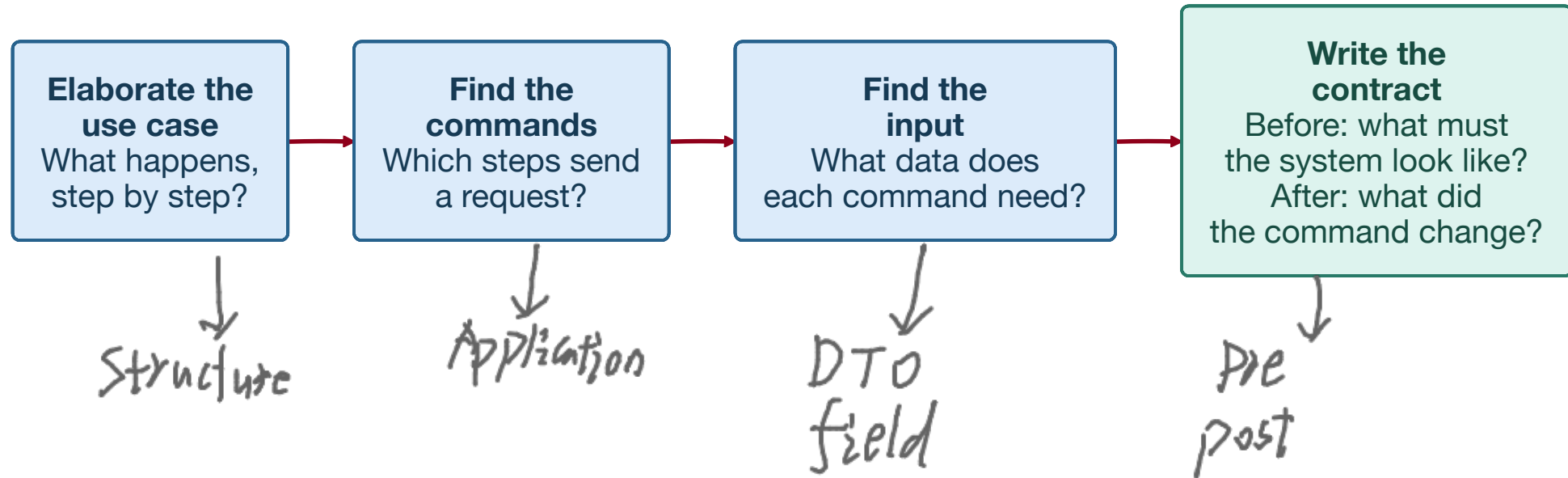
FALL 2026

SEG3102 Software Design and Architecture

Domain-Driven Design: From Contracts to Structure Design

Slide content used in this course is adapted from Prof. Stéphane Somé.

Last Lecture: Behavior design steps



Behavior-driven development (BDD)

- Behavior focused tests: what
 - human understandable language
- BDD tool automates matching to code
 - example: Cucumber (<https://cucumber.io>)
 - scenarios grouped in features
 - defined in Gherkin syntax

Last Lecture: Mapping between contracts and scenarios

Contract

Preconditions:

- signed-in `customer` (non null)
- the `customer's` `cart` has at least one `cart line`
- for each `cart line`, the available stock covers the `quantity`

Command: `placeOrder(customer, orderInfo)`

Post-conditions:

- `ord` was added to the `customer's` orders
- the `available stock` of each product was reduced by the `ordered quantity`
- the `cart lines` were removed from the cart

Scenario

Given a signed-in customer
with 2 "Wireless mouse" in the cart
And 10 "Wireless mouse" available in stock

When the customer places the order

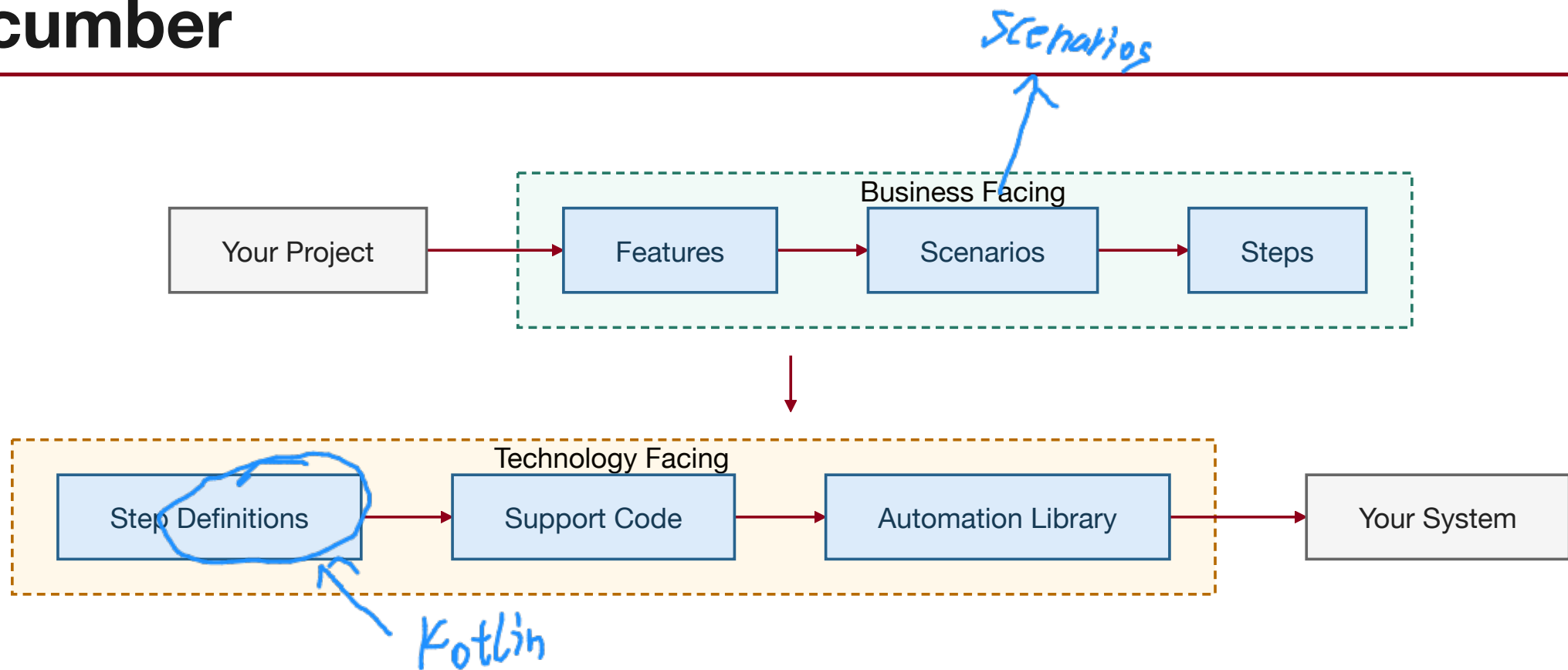
Then a new order is added to the customer's orders
And 8 "Wireless mouse" remain available in stock
And the cart is empty

Outline

- BDD contracts for the online auction system
- Structure design: Modeling domain concepts and their relationships

BDD for the online auction system

Cucumber



Two feature creation strategy

Explicit data, implicit behavior

```

1 Scenario Outline: Ordering depends on the available stock
2   Given a signed-in customer with <in cart> <item> in the cart
3   And <available> <item> available in stock
4   When the customer places the order
5   Then the order is <outcome>
6   And <remaining> <item> remain available in stock
7
8   Examples:
9     | in cart | available | outcome   | remaining | item           |
10    | 2       | 10       | placed    | 8         | Wireless mouse |
11    | 5       | 5        | placed    | 0         | Wireless mouse |
12    | 6       | 5        | not placed | 5         | Wireless mouse |

```

Implicit data (data hidden in code), explicit behavior

Scenario: The available stock covers the ordered quantity

- Given** a signed-in customer with items in the cart
- And** the available stock covers the quantity of each item
- When** the customer places the order
- Then** the order is placed
- And** the available stock of each item is reduced by the ordered quantity

Scenario: The available stock does not cover the ordered quantity

- Given** a signed-in customer with items in the cart
- And** the available stock does not cover the quantity of an item
- When** the customer places the order
- Then** the order is not placed
- And** the available stock is unchanged

Feature

Command: addAuction(seller, auctionInfo)

Preconditions:

- signed-in seller (non null)
- non-existing Pending Payment for Seller

Post-conditions:

- a new Auction auc was created
- auc was initialized from auctionInfo
- auc was added to seller auctions
- new Auction Item aucIt was created
- aucIt was initialized from auctionInfo
- aucIt was set as item for auc
- If credit card infos were provided in auctionInfo
 - a new Credit Card ccard was created
 - ccard was initialized from credit card infos
 - ccard was set as seller's credit card
 - ccard was set as credit card for auc

1 **Feature:** Add new auction by Seller.

A scenario for each unique setup

Command: addAuction(seller, auctionInfo)

Preconditions:

- **signed-in seller (non null)**
- **non-existing Pending Payment for Seller**

Post-conditions:

- a new Auction auc was created
- auc was initialized from auctionInfo
- auc was added to seller auctions
- new Auction Item aucIt was created
- aucIt was initialized from auctionInfo
- aucIt was set as item for auc
- **If credit card infos were provided in auctionInfo**
 - a new Credit Card ccard was created
 - ccard was initialized from credit card infos
 - ccard was set as *seller's* credit card
 - ccard was set as credit card for auc

```

1 Feature: Add new auction by Seller.
2   Scenario: Seller is signed in,
3     Seller has no Pending Payment,
4     Auction information doesn't
5       include Credit card information
6   ....
7   Scenario: Seller is signed in,
8     Seller has no Pending Payment,
9     Auction information includes Credit card
10    information
  
```

Preconditions & alternatives



Preconditions & alternative conditions

Command: addAuction(seller, auctionInfo)

Preconditions:

- **signed-in seller (non null)**
- **non-existing Pending Payment for Seller**

Post-conditions:

- a new Auction auc was created
- auc was initialized from auctionInfo
- auc was added to seller auctions
- new Auction Item aucIt was created
- aucIt was initialized from auctionInfo
- aucIt was set as item for auc
- **If credit card infos were provided in auctionInfo**
 - a new Credit Card ccard was created
 - ccard was initialized from credit card infos
 - ccard was set as *seller's* credit card
 - ccard was set as credit card for auc

```

1 Feature: Add new auction by Seller.
2   Scenario: ...
3     Given the seller is signed in
4     And the seller has no pending payment
5     And auction information is provided
6     And the auction information
7       does not include credit card information
8     ...
9   Scenario: ...
10    Given the seller is signed in
11    And the seller has no pending payment
12    And auction information is provided
13    And the auction information includes credit
14      card information
15    ...
  
```

Application command invocation

Command: addAuction(seller, auctionInfo)

Preconditions:

- signed-in seller (non null)
- non-existing Pending Payment for Seller

Post-conditions:

- a new Auction auc was created
- auc was initialized from auctionInfo
- auc was added to seller auctions
- new Auction Item aucIt was created
- aucIt was initialized from auctionInfo
- aucIt was set as item for auc
- If credit card infos were provided in auctionInfo
 - a new Credit Card ccard was created
 - ccard was initialized from credit card infos
 - ccard was set as *seller's* credit card
 - ccard was set as credit card for auc

```

1  Feature: Add new auction by Seller.
2  Scenario: ...
3    Given the seller is signed in
4    And the seller has no pending payment
5    And auction information is provided
6    And the auction information
7      does not include credit card information
8    When the application command addAuction is
9      invoked
10 Scenario: ...
11 Given the seller is signed in
12 And the seller has no pending payment
13 And auction information is provided
14 And the auction information includes credit
15   card information
16 When the application command addAuction is
17   invoked
  
```

Postconditions

Command: addAuction(seller, auctionInfo)

Preconditions:

- signed-in seller (non null)
- non-existing Pending Payment for Seller

Post-conditions:

- **a new Auction auc was created**
- **auc was initialized from auctionInfo**
- **auc was added to seller auctions**
- **new Auction Item aucIt was created**
- **aucIt was initialized from auctionInfo**
- **aucIt was set as item for auc**
- If credit card infos were provided in auctionInfo
 - a new Credit Card ccard was created
 - ccard was initialized from credit card infos
 - ccard was set as *seller's* credit card
 - ccard was set as credit card for auc

```

1  Feature: Add new auction by Seller.
2  Scenario: ...
3    Given ...
4    When the application command addAuction is invoked
5    Then a new auction is created
6    And the new auction is initialized from the
7      auction information
8    And the new auction is added to the seller's
9      auctions
10   And a new item is created
11   And the new item is initialized from the
12     auction information
13   And the new auction is linked to the new item
14 Scenario: ...
15   ...
  
```

Postconditions

Command: addAuction(seller, auctionInfo)

Preconditions:

- signed-in seller (non null)
- non-existing Pending Payment for Seller

Post-conditions:

- a new Auction auc was created
- auc was initialized from auctionInfo
- auc was added to seller auctions
- new Auction Item aucIt was created
- aucIt was initialized from auctionInfo
- aucIt was set as item for auc
- **If credit card infos were provided in auctionInfo**
 - **a new Credit Card ccard was created**
 - **ccard was initialized from credit card infos**
 - **ccard was set as seller's credit card**
 - **ccard was set as credit card for auc**

```

1 Feature: Add new auction by Seller.
2   Scenario: ...
3   Scenario: ...
4   ...
5   Then a new auction is created
6   And the new auction is initialized from the
7     auction information
8   And the new auction processing fee has been
9     set
10  And the new auction is added to the seller's
11    auctions
12  And a new item is created
13  And the new item is initialized from the
14    auction information
15  And the new auction is linked to the new item
16  And a new credit card is created
17  And the new credit card is initialized from the
18    credit card information
19  And the new credit card is set as the seller's
20    credit card

```

The feature file in the tutorial project

src/test/resources/seg3x02.auctionsystem.contracts/add_auction.feature

```
1 Feature: Add new auction by Seller.
2   Scenario 1: Seller is signed in, Seller has no Pending Payment, Auction information doesn't include Credit card information
3     Given the seller is signed in
4     And the seller has no pending payment
5     And auction information is provided
6     And the auction information does not include credit card information
7     When the application command AddAuction is invoked
8     Then a new auction is created with attributes matching the auction information
9     And the new is added to the seller's auctions
10    And a new item is created with attributes matching the auction information
11    And the new auction is linked to the new item
12  Scenario 2: Seller is signed in, Seller has no Pending Payment, Auction information includes Credit card information
13    Given the seller is signed in
14    And the seller has no pending payment
15    And auction information is provided
16    And the auction information includes credit card information
17    When the application command AddAuction is invoked
18    Then a new auction is created with attributes matching the auction information
19    And the new is added to the seller's auctions
20    And a new item is created with attributes matching the auction information
21    And the new auction is linked to the new item
22    And a new credit card is created with attributes matching the credit card information
23    And the new credit card is associated to the new seller
24    And the new credit card is associated to the new auction
```

Tutorial repository

<https://github.com/SEG3102-Fall2026/auction-system-tutorials>

Branch: `contracts`

Exercise: Scenarios of Close Auction

Close Auction runs when an auction's end time is reached.

Command: `closeAuction(auctionId)`

Notes:

- check if there were bids for the auction
- submit the transaction fee payment request to the Credit Service Bureau
- send emails to the Buyer and Seller

Preconditions:

- there is an existing auction identified by `auctionId`
- auction is open
- auction end time has been reached

Post-conditions:

- auction was marked as closed (modification of attributes)
- **If** the transaction fee payment was not successful
 - a pending payment was added to the Seller's account (modification of attributes)

We now write the feature for `closeAuction`. How many scenarios does it need?

Exercise: Scenarios of Close Auction

Go to wooclap.com and use the code **FNXKRSF**



How many scenarios does the feature for closeAuction need?

- 1

1: the auction closes

3% 1
- 2

2: the fee payment succeeds, or it fails

✓ Correct answer
61% 23
- 3

2: the fee payment succeeds, and no bids

5% 2
- 4

4: one for each precondition, plus the main scenario

32% 12

Exercise: Answers

```

1 Command: closeAuction(auctionId)
2
3 Notes:
4   - check if there were bids for the auction
5   - ...
6
7 Preconditions:
8   - there is an existing auction ...
9   - auction is open
10  - auction end time has been reached
11
12 Post-conditions:
13   - auction was marked as closed
14   - If the transaction fee payment was not
15     successful
16   - a pending payment was added to the
17     Seller's account

```

close_auction.feature

```

Feature: Close auction at deadline
  Scenario: Successful auction fee payment
    ...
  Scenario: Unsuccessful auction fee payment
    ...

```

Two scenarios. The preconditions are the same in both. The only branch is the result of the fee payment, determined by the post conditions.

- The bids are in the notes. No post-condition depends on them.
- A contract assumes its preconditions hold. When one is false, the command is not invoked.

Exercise: Where does this step come from?

Command: `closeAuction(auctionId)`

Notes:

- check if there were bids for the auction
- submit the transaction fee payment
 request to the Credit Service Bureau
- send emails to the Buyer and Seller

Preconditions:

- there is an existing auction
 identified by `auctionId`
- auction is open
- auction end time has been reached

Post-conditions:

- auction was marked as closed
- **If** the transaction fee payment was
 not successful
 - a pending payment was added to the
 Seller's account

`close_auction.feature`

```

1 Feature: Close auction at deadline
2   Scenario: Successful auction fee payment
3     Given the auction is open
4     And the auction deadline has passed
5     And the auction seller credit card is able to settle the auction fee
6     When the application command closeAuction is invoked
7     Then the auction is marked as closed
8   Scenario: Unsuccessful auction fee payment
9     Given the auction is open
10    And the auction deadline has passed
11    And the auction seller credit card is unable to settle the auction fee
12    When the application command closeAuction is invoked
13    Then the auction is marked as closed
14    And the auction fee is set as pending payment to the
15      auction seller account

```

Line 11 is a **Given**. Which part of the contract does it come from?

Exercise: Where does this step come from?

Go to wooclap.com and use the code **FNXKRSF**

Line 11 ("And the auction seller credit card is unable to settle the auction fee") is a Given. Which part of the contract does it come from?

- 1

A precondition

50% 20
- 2

A postcondition

✓ Correct answer

48% 19
- 3

A note

0% 0
- 4

None: it is extra setup added by the tester

3% 1

Exercise: Answers

```

1 Command: closeAuction(auctionId)
2
3 Notes:
4   - check if there were bids for the auction
5   - submit the transaction fee payment
6     request to the Credit Service Bureau
7   - send emails to the Buyer and Seller
8
9 Preconditions:
10  - there is an existing auction
11    identified by auctionId
12  - auction is open
13  - auction end time has been reached
14
15 Post-conditions:
16  - auction was marked as closed
17  - If the transaction fee payment was
18    not successful
19    - a pending payment was added to the
20      Seller's account

```

close_auction.feature

```

1 Feature: Close auction at deadline
2   Scenario: Successful auction fee payment
3     Given the auction is open
4     And the auction deadline has passed
5     And the auction seller credit card is able to settle the auction fee
6     When the application command closeAuction is invoked
7     Then the auction is marked as closed
8   Scenario: Unsuccessful auction fee payment
9     Given the auction is open
10    And the auction deadline has passed
11    And the auction seller credit card is unable to settle the auction fee
12    When the application command closeAuction is invoked
13    Then the auction is marked as closed
14    And the auction fee is set as pending payment to the
15      auction seller account

```

From the condition in the post-conditions. A **Given** sets up the state before the command, so it comes from the preconditions and from the alternative conditions. Line 5 is the other branch of the same condition.

Structure Design: Modeling domain concepts and their relationships

What the contract gives us

Command: `placeOrder(customer, orderInfo)`

Preconditions:

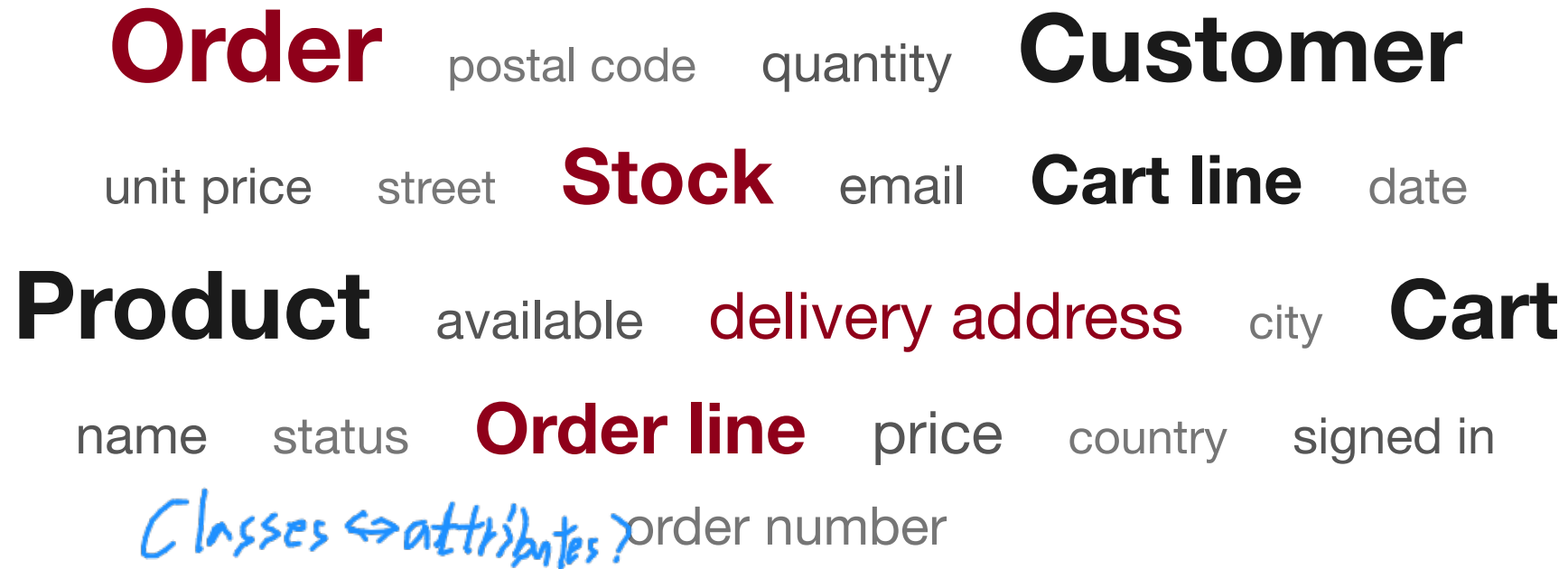
- signed-in `customer` (non null)
- the `customer's` `cart` has at least one `cart line`
- for each `cart line`, the `available stock` covers the `quantity`

Post-conditions:

- a new Order `ord` was created (creation of instance)
- `ord` was initialized from `orderInfo` (modification of attributes)
- `ord` was added to the `customer's` `orders` (creation of association)
- for each `cart line`, a new Order Line `ln` was created from it (creation of instance, modification of attributes)
- `ln` was added to `ord` (creation of association)
- the `available stock` of each product was reduced by the ordered `quantity` (modification of attributes)
- the `cart lines` were removed from the `cart` (destruction of instances)

- Tests through behavior-driven development
- **Structure modeling of the domain**
 - Every noun is a candidate concept: Customer, Cart, Cart Line, Order, Order Line, Product, Stock.

What do we need to store?



These are data we potentially need to store.
 How should we represent them in the code?

Entity

Identity

Customer
name email

Order
date deliveryAddress total

OrderLine
product quantity unitPrice

Two customers have the same name. Which one placed this order?

A customer places the same order twice: same products, same address. Which one did we ship?

A customer changes the quantity of a line from 2 to 3. Is it still the same line?

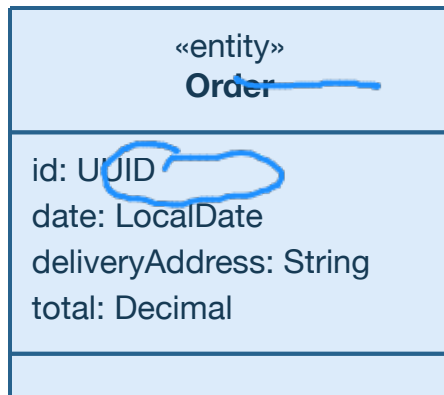
→ A separate identity

The attributes cannot tell them apart, and the attributes change. Each one needs an identity of its own.

Entity

Entity

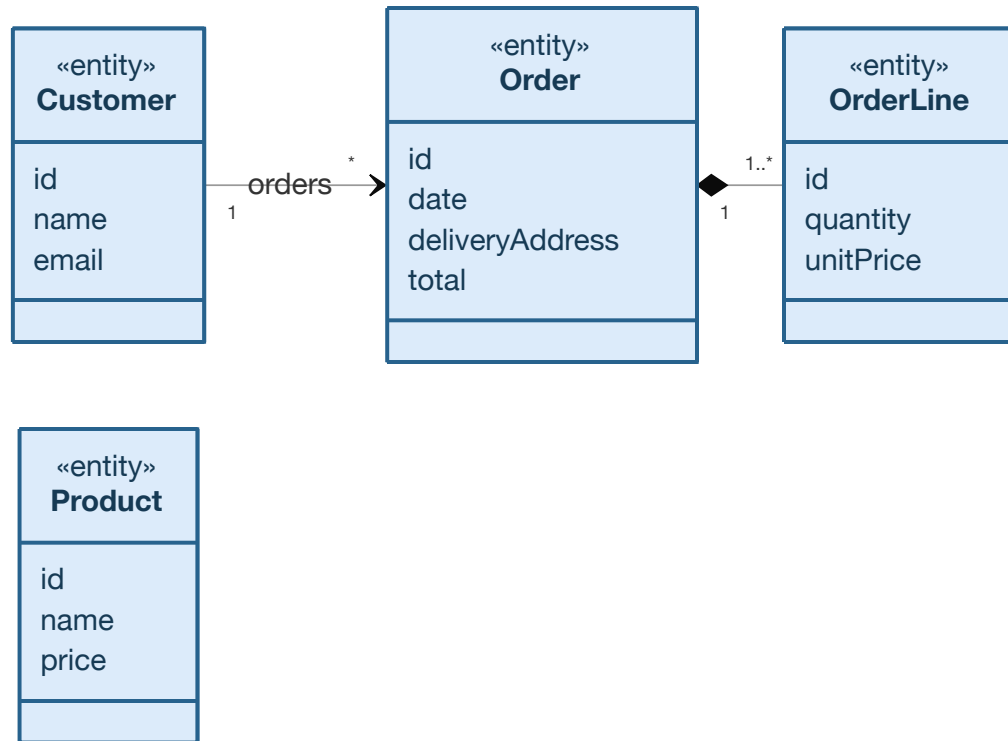
An **entity** is a domain object defined by its identity. Its attributes can change during its life; its identity never changes.



```
1 class Order(  
2     val id: UUID,  
3     val date: LocalDate,  
4     var deliveryAddress: String,  
5     var total: BigDecimal  
6 ) {  
7     override fun equals(other: Any?) =  
8         other is Order && other.id == id  
9  
10    override fun hashCode() = id.hashCode()  
11 }
```

Equality is based on the **id**. Two orders with the same attributes are still two distinct orders, each with its own id.

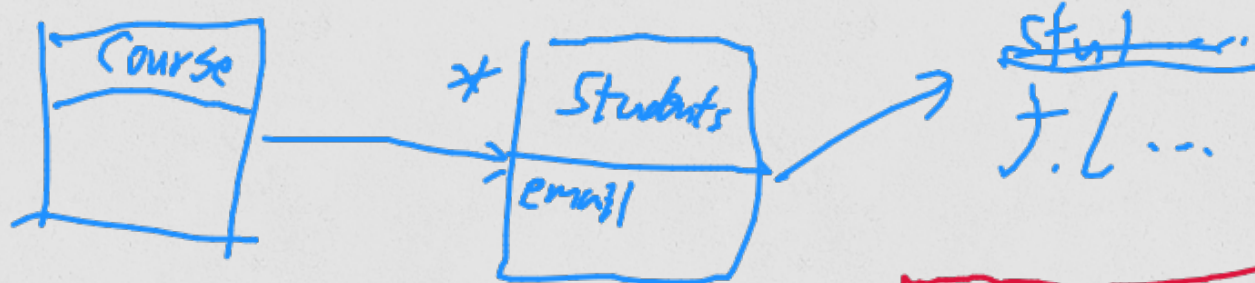
Entities in the online shop



An order is tracked from checkout to delivery. A product changes its price over time. Both stay the same object while their attributes change.

Test: when every attribute changes, should they still be treated as the same object? If so, Then it is an entity.

Q: Can ~~email~~ be the identify for a Customer?



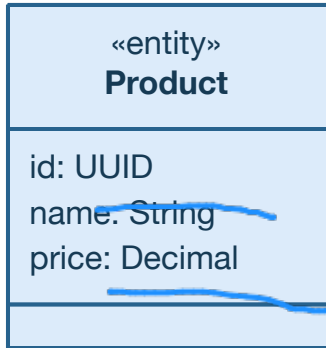
Name	Stu_email
Seg3/02	Stu1. Lottmann

Conditions of identity

- ① Unique
- ② Stable

Value object

20 what?



Good news: we now sell to the US and to Europe.
Prices are in Canadian dollars, US dollars, or euros.

Oh no, the price is just a number. The wireless
mouse costs 20. 20 what?



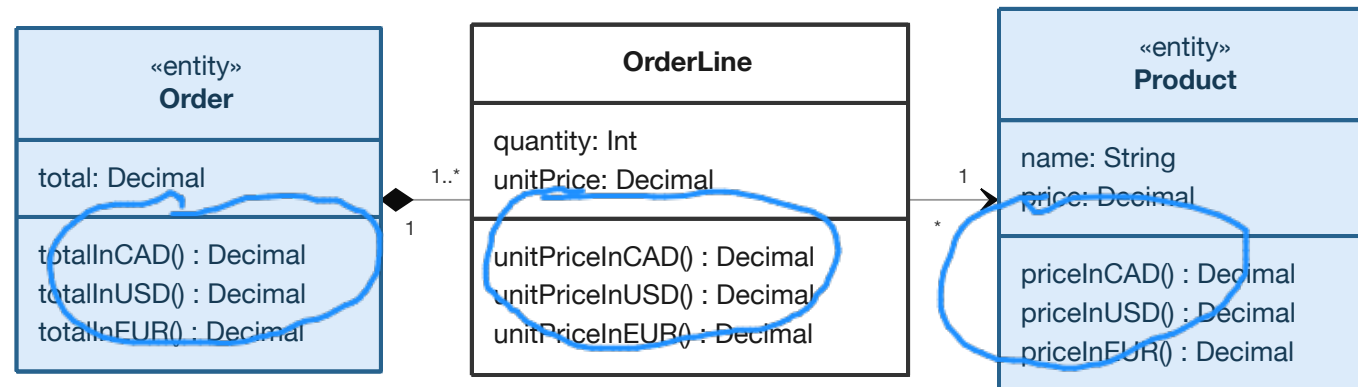
OK, you'd better fix this!

Can we survive this?



Alright, if we assume the price is stored in CAD by default, I should survive this with conversion methods!

```
1 // Don't forget the prices are stored in CAD!!!
2 class Product(val id: UUID, var name: String, var price: BigDecimal) {
3     ...
4 }
```



The same three methods, in three classes

Some helper methods...

```
1 // prices are stored in CAD
2 object CurrencyConverter {
3     fun cadToUsd(amount: BigDecimal): BigDecimal =
4         amount * ExchangeRateService.rate("CAD", "USD")
5
6     fun cadToEur(amount: BigDecimal): BigDecimal =
7         amount * ExchangeRateService.rate("CAD", "EUR")
8 }
9
10 class Product(val id: UUID, var name: String, var price: BigDecimal) {
11     fun priceInCAD() = price
12     fun priceInUSD() = CurrencyConverter.cadToUsd(price)
13     fun priceInEUR() = CurrencyConverter.cadToEur(price)
14 }
```

Singleton class

class functions

handle price
↳ CurrencyConverter

We are going BIG



Next week, we will open in Japan!

Then `...InJPY()` in *Product*, *OrderLine*, *Order*, and in every class that has a price.



... several months later...



We are opening in 10 more countries! Moreover, we will allow customers to buy products not in their own currency as well!

20 USD + 15CAD

OK, OK...



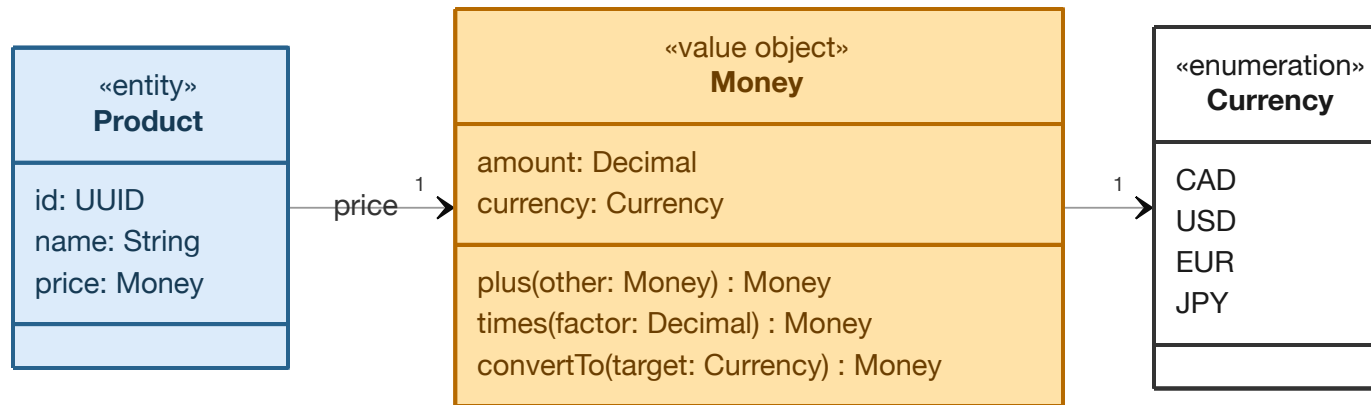
We want to avoid this!



Value object

Value object

A **value object** is a domain object defined only by its attributes. It has no identity and it never changes: to change a value, replace it with a new one.



Product, OrderLine, and Order all use **Money**. A new currency is one more value in **Currency**.

We can also use value object to implement business logic!

Money in code

```

1  enum class Currency { CAD, USD, EUR, JPY }
2
3  data class Money(val amount: BigDecimal, val currency: Currency) {
4      operator fun plus(other: Money): Money {
5          val converted = other.convertTo(currency)
6          return Money(amount + converted.amount, currency)
7      }
8
9      operator fun times(factor: BigDecimal) =
10         Money(amount * factor, currency)
11
12     fun convertTo(target: Currency) =
13         Money(amount * ExchangeRateService.rate(currency, target), target)
14 }

```

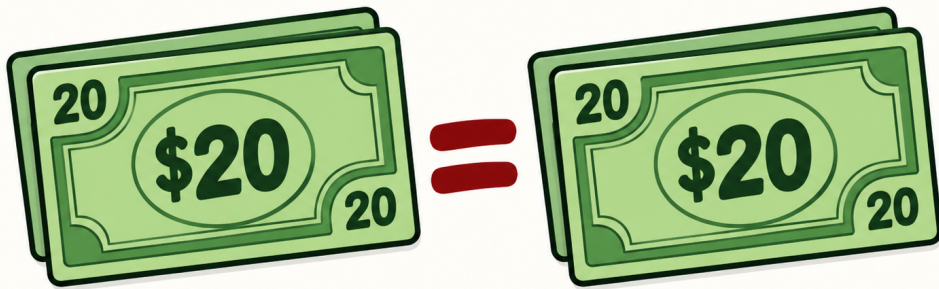
*a: Money
b: Money*

→ a + b

*→ 4 * a*

20 CAD + 15 USD now converts the USD first and gives an amount in CAD. The rule is in one place.

Equal by value and never changed



val → constant
var → variable

Two \$20 bills are interchangeable. Nobody asks which one they got.

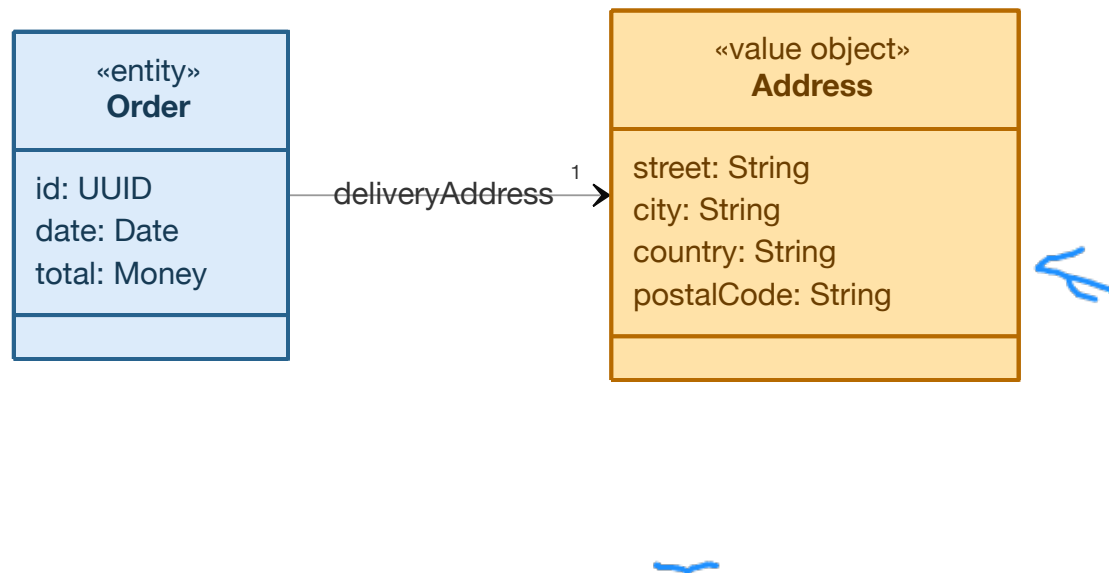
```
1 val a = Money(BigDecimal(20), CAD)
2 val b = Money(BigDecimal(20), CAD)
3 a == b                                // true
4           
5 // a price change replaces the value
6 mouse.price = Money(BigDecimal(25), CAD)
```

A Kotlin data class with only val *constant* properties is equal by value and immutable.

Value objects have no identity!

The delivery address

Order information: the customer placing the order and the delivery address (street, city, country, postal code).



A customer moves. Should the address of their order from last year change too?

No. The order keeps the address it was shipped to. Their account should get a new **Address**.

Value obj

attr

① Composite

② Custom operations

price . float

total float

discount float

Money

Person

nextTo . string

nextTo . Person



The modeling choice depends on the context

Online shop

An address describes where to ship.
Two orders to the same street, city,
and postal code go to the same place.

Value object

Postal service

An address points to a physical
location that is tracked over time: it is
created when a building is built, and
renamed when a street is renamed.

Entity

Entity or value object is not a property of the
concept. It depends on the bounded context.

Exercise: Entity or value object?

Which of these are best modeled as value objects? Select all that apply.

- A. A bank account in a banking app, whose balance changes with every deposit
- B. The version number of a software release, e.g., 2.4.1
- C. A patient in a hospital system, whose address and phone number change over the years
- D. A colour in a drawing app, e.g., RGB (143, 0, 26)

Exercise: Entity or value object?

Go to wooclap.com and use the code **FNXKRSF**



Multiple-answer question. Which of these are best modeled as value objects?

- ① A bank account in a banking app, whose balance changes with every deposit

27% 9
- ② The version number of a software release, e.g., 2.4.1

✓ Correct answer

45% 15
- ③ A patient in a hospital system, whose address and phone number change over the years

24% 8
- ④ A colour in a drawing app, e.g., RGB (143, 0, 26)

✓ Correct answer

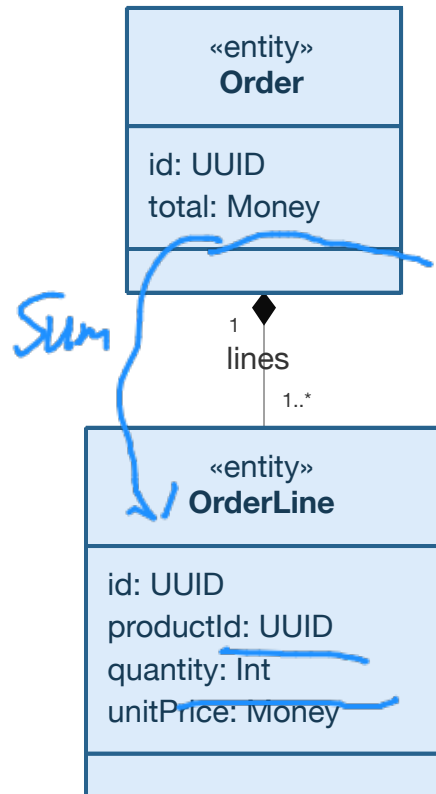
70% 23

Exercise: Answers

- A. Bank account: entity. Its balance changes, and the account number identifies it.
- **B. Version number: value object. Two releases numbered 2.4.1 have the same version; the next release gets a new number, 2.4.2, instead of changing the old one.**
- C. Patient: entity. The address and phone number change, and it is still the same patient.
- **D. Colour: value object. Two shapes filled with RGB (143, 0, 26) have the same colour; recolouring a shape gives it a new colour value.**

Aggregate

Customers want to change their order



Customers keep asking if they can change the quantity before their order ships?

Easy. An order line has a quantity; the **ChangeOrder** use case sets it.



```

1 // in the ChangeOrder use case
2 val line = order.lines.first { it.id == lineId }
3 line.quantity = 3
  
```

Something does not add up



Alright, let's test the `ChangeOrder` use case with a Gherkin scenario.

```
1 Scenario: The customer changes the quantity of an order line
2   Given an order with 2 "Wireless mouse" at 20 dollars each
3   And the order total is 40 dollars
4   When the customer changes the quantity of "Wireless mouse" to 3
5   Then the order has 3 "Wireless mouse"
6   And the order total is 60 dollars
```

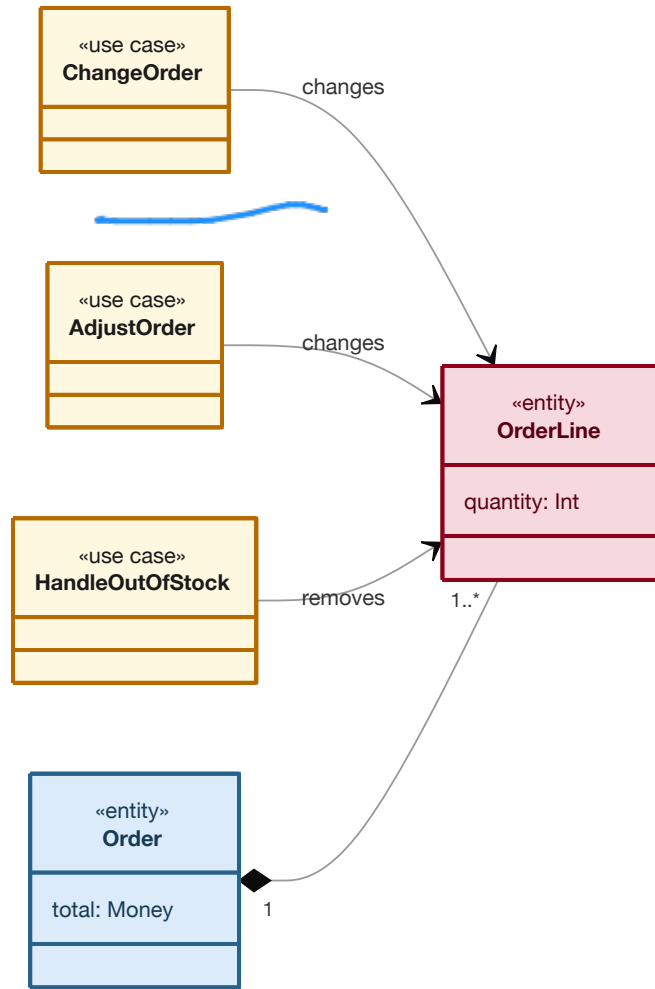
The last step fails: the total is still 40 dollars. Good thing the scenario caught it before an invoice went out.

```
1 // in the ChangeOrder use case
2 val line = order.lines.first { it.id == lineId }
3 line.quantity = 3
4 order.total = order.lines.map { it.subtotal() }.reduce(Money::plus)
```



Fixed, in `ChangeOrder`. The scenario passes.

A few months later



Three use cases now change order lines...



One more thing: an order always has at least one line.

Three use cases must update the total and check that a line is left. Did anyone write one for [HandleOutOfStock](#)?



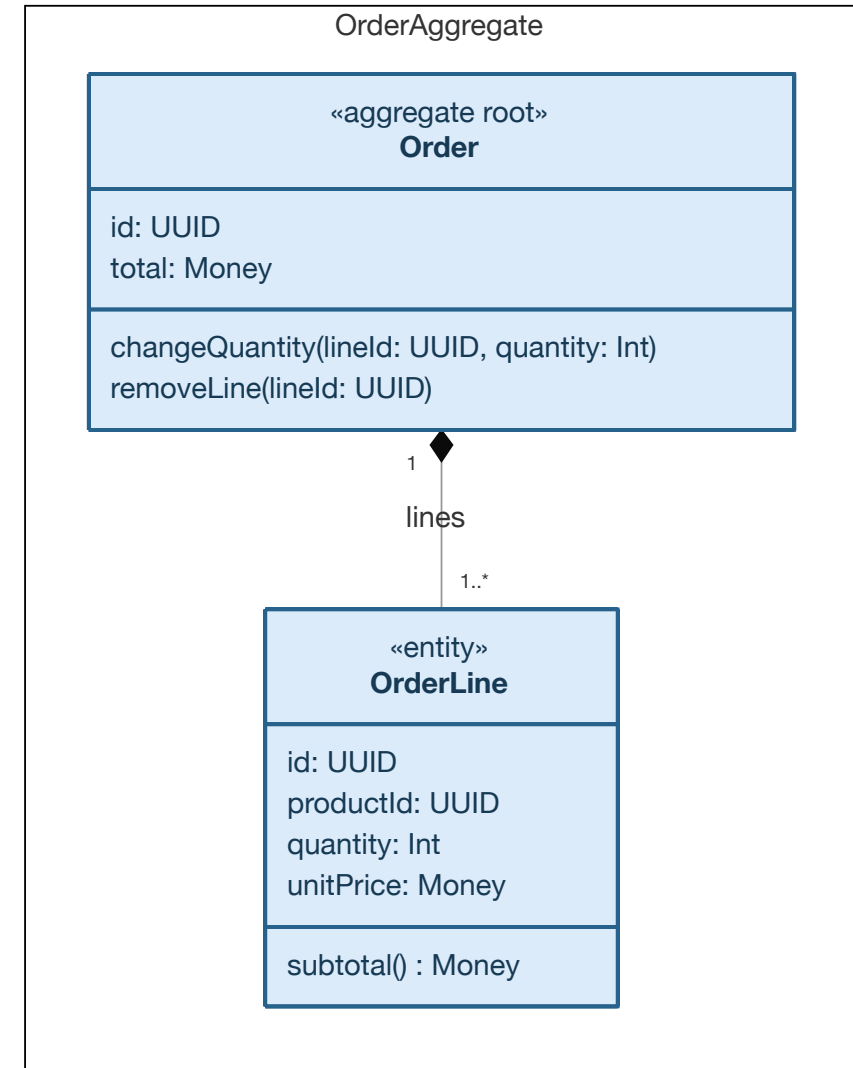
I'm pretty sure there will be more use cases like these coming... The rules belong to the order, but every code that touches an order line has to enforce them...



Aggregate

Aggregate

An **aggregate** is a cluster of entities and value objects that change together. One entity, the **aggregate root**, is the only entry point: outside code changes the cluster only through the root, and the root keeps the whole cluster consistent with its rules.



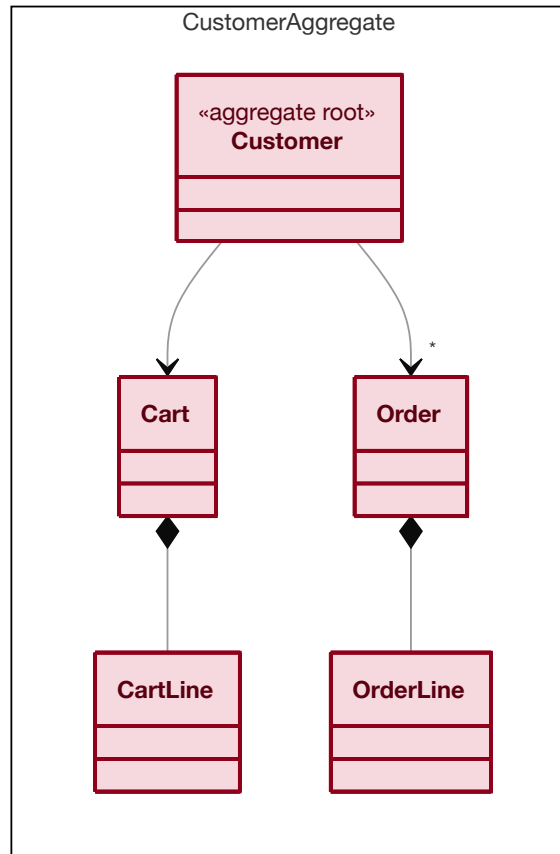
Change only through the root

```
1 class Order(val id: UUID, /* ... */ lines: List<OrderLine>) {
2     private val _lines = lines.toMutableList()
3     val lines: List<OrderLine> get() = _lines    // read-only view
4
5     init {
6         require(_lines.isNotEmpty()) { "An order has at least one line" }
7     }
8     var total: Money = sum(); private set
9
10    fun changeQuantity(lineId: UUID, quantity: Int) {
11        _lines.first { it.id == lineId }.quantity = quantity // internal setter
12        total = sum()
13    }
14
15    fun removeLine(lineId: UUID) {
16        require(_lines.size > 1) { "An order has at least one line" }
17        _lines.removeIf { it.id == lineId }
18        total = sum()
19    }
20
21    private fun sum() = _lines.map { it.subtotal() }.reduce(Money::plus)
22 }
```

The use cases can read the order lines, but every change goes through `Order`, which keeps the total right and the order non-empty.

Why not one big aggregate?

Customer owns the cart and the orders. Let's make Customer the root of everything: all the rules in one place.



Technically, this is not wrong. But now...

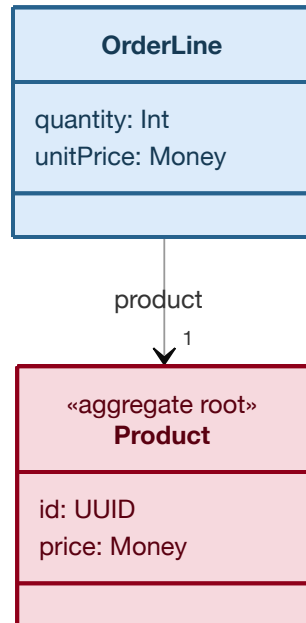
Loading a customer loads every order they placed in ten years.

If the operations are synchronous, while the warehouse updates one of their orders, adding a mouse to their cart has to wait.

Keep aggregates small: an aggregate holds only what must be consistent at the same moment.

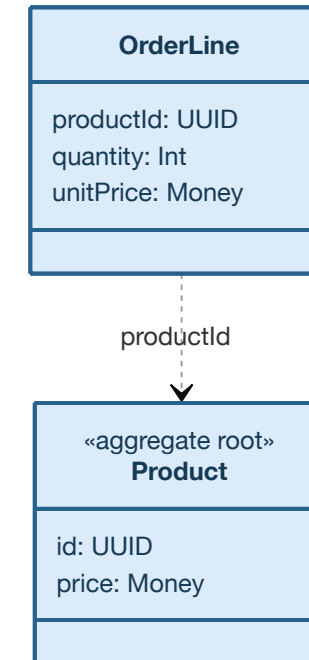
Reference other aggregates by id

Object reference



Code holding an order can change the product.
Loading an order also loads all product information.

Identity reference



The order knows which product, and nothing more.
Each aggregate is loaded and changed on its own.

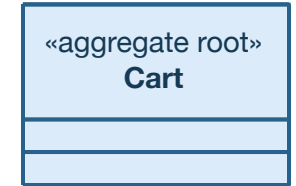
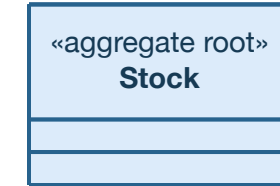
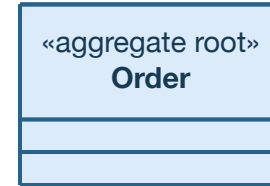
Prefer identity references between aggregates: hold the id of the other root, not the object reference.

One transaction, one aggregate

The contract of `placeOrder` changes three aggregates:

Post-conditions:

- a new Order `ord` was created
- ...
- the `available stock` of each product was reduced by the `ordered quantity`
- the `cart lines` were removed from the cart



Since aggregate is the consistency boundary, one transaction changes one aggregate. So how do Stock and Cart learn that an order was placed?

The answer is: through event notifications.

Summary

- Two ways to write a **feature**: explicit data in a scenario outline, or explicit behavior with the data hidden in the step definitions.
- An **entity** is defined by its identity, which never changes while its attributes do.
- A **value object** is defined only by its attributes and never changes. It can hold business logic, such as adding and converting **Money**.
- An **aggregate** is a cluster of entities and value objects that change together, only through the **aggregate root**. Keep aggregates small, and reference other aggregates by id.